Full Stack Developer Assessment

Assignment: Transaction Ledger API
Duration: 3-4 hours

Objective
Build a backend API for a simple transaction ledger system.
The system should allow users to create accounts, perform credit and debit transactions, and retrieve account balances and transaction history.

1. Create Account
Create an API to create a new account.
Each account must contain:

- id — unique account identifier
- name — account holder name
- balance — current account balance
- created_at — account creation timestamp

Requirements

- id must be unique.
- name is required.
- Initial balance must always be 0.
- created_at should be generated by the system.
- The client should not be allowed to set the initial balance.

2. Add Transaction
Create an API to add a transaction to an account.
Each transaction must contain:

- id — unique transaction identifier
- account_id — ID of the account
- type — credit or debit
- amount — transaction amount
- description — description of the transaction
- reference_id — unique reference for the transaction within an account
- created_at — transaction creation timestamp

Transaction Rules
Credit
A credit transaction must increase the account balance by the transaction amount.
Example:
Current balance: 100
Credit: 50
New balance: 150

Full Stack Developer Assessment

Debit
A debit transaction must decrease the account balance by the transaction amount.
Example:
Current balance: 100
Debit: 30
New balance: 70

A debit transaction must be rejected if the account does not have sufficient balance.
Example:
Current balance: 100
Debit: 150

Result: Transaction rejected
Balance remains: 100

Duplicate Transactions
For the same account, reference_id must be unique.
If a transaction with the same reference_id is submitted again for the same account:

- A new transaction must not be created.
- The account balance must not change.
- The API should return an appropriate response indicating that the transaction already exists.

The same reference_id may be used for different accounts.

3. Get Account Balance
Create an API to fetch the current balance of an account.
The response should include at least:

- account_id
- balance

If the account does not exist, return an appropriate error response.

4. Get Transaction History
Create an API to fetch all transactions for an account.
The API must support:
Filtering
Filter transactions by type:
credit
debit

If no type is provided, return both credit and debit transactions.t.

Full Stack Developer Assessment

Pagination
Support pagination using parameters such as:
page
limit

The API should return enough information for the client to understand the pagination result, such as:

- current page
- page size
- total records
- total pages

Sorting
Transactions must be returned in **latest-first order**, based on created_at.

5. Data Integrity
The implementation must maintain data consistency at all times.
Transaction Immutability
Once a transaction is created, it must not be editable or deletable through the API.
The transaction's:

- type
- amount
- description
- reference ID
- account ID
- creation timestamp

must remain unchanged.

Balance Consistency
The account balance must always represent the result of all valid transactions for that account.
For example:
Initial balance: 0

Credit: 100
Credit: 50
Debit: 30
Debit: 20

Final balance: 100

A failed or rejected transaction must not affect the account balance.

Full Stack Developer Assessment

Atomicity
Creating a transaction and updating the account balance must be treated as one atomic operation.
If either operation fails, neither change should be persisted.

7. Validations
The API should validate all incoming data.
At minimum:

- Account name must be provided.
- Account ID must exist when creating a transaction.
- Transaction type must be either credit or debit.
- Amount must be greater than 0.
- reference_id must be provided.
- Duplicate reference_id for the same account must be handled correctly.
- Debit transactions must have sufficient balance.
- Invalid account IDs should return an appropriate error.
- Invalid request data should return an appropriate validation error.

8. Error Handling
Implement proper HTTP status codes and meaningful error responses.
Examples of cases that should be handled:

- Account not found
- Invalid transaction type
- Invalid amount
- Insufficient balance
- Duplicate transaction
- Invalid pagination parameters
- Invalid request body

Error responses should be consistent and understandable.

9. Bonus: Money Transfer
If time permits, implement an API to transfer money from one account to another.
Example:
Account A balance: 100
Account B balance: 50

Transfer: 30 from A to B

Account A balance: 70
Account B balance: 80

Full Stack Developer Assessment

Transfer Requirements

- Source account must have sufficient balance.
- Both accounts must exist.
- The debit from the source account and credit to the destination account must happen atomically.
- If any part of the transfer fails, the entire transfer must be rolled back.
- A partial transfer must never be possible.

10. Technical Requirements

- Use any backend technology/framework of your choice.
- Use any relational or non-relational database of your choice.
- Follow a clean and maintainable project structure.
- Implement proper validation.
- Implement proper error handling.
- Use database constraints where appropriate to maintain data integrity.
- The application should be runnable locally using the instructions provided in the README.

11. Submission

1. Source Code
2. README
3. API Documentation
4. Assumptions